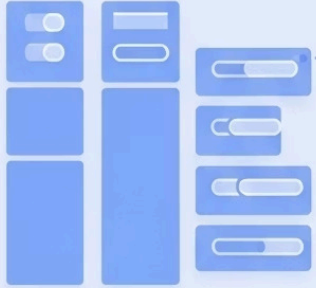
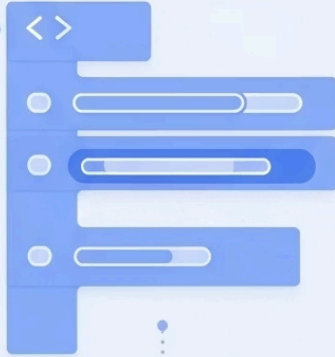


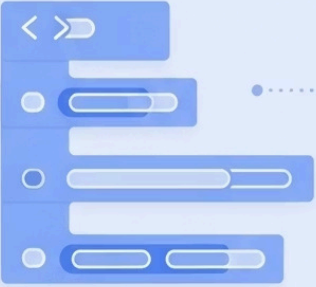
display: block;



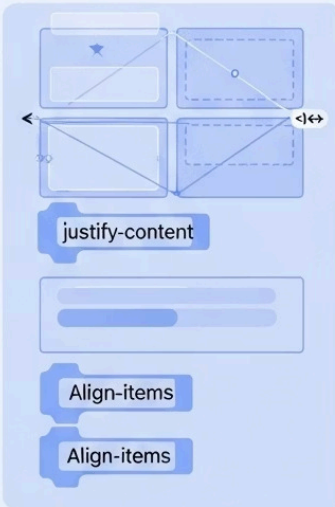
display: inline;



display: inline-block;



display: flex;



display: flex;



Display y Visibilidad en CSS

El CSS no solo da color o estilos, también define cómo se comportan los elementos en la página. Controlar la forma en que los elementos se muestran y su visibilidad es fundamental para crear diseños web atractivos y funcionales.

Display: Block

Los elementos `block` ocupan todo el ancho disponible de su contenedor y comienzan en una nueva línea, independientemente de su contenido.

Ejemplos clásicos:

- Párrafos `<p>`
- Encabezados `<h1>` a `<h6>`
- Divisiones `<div>`

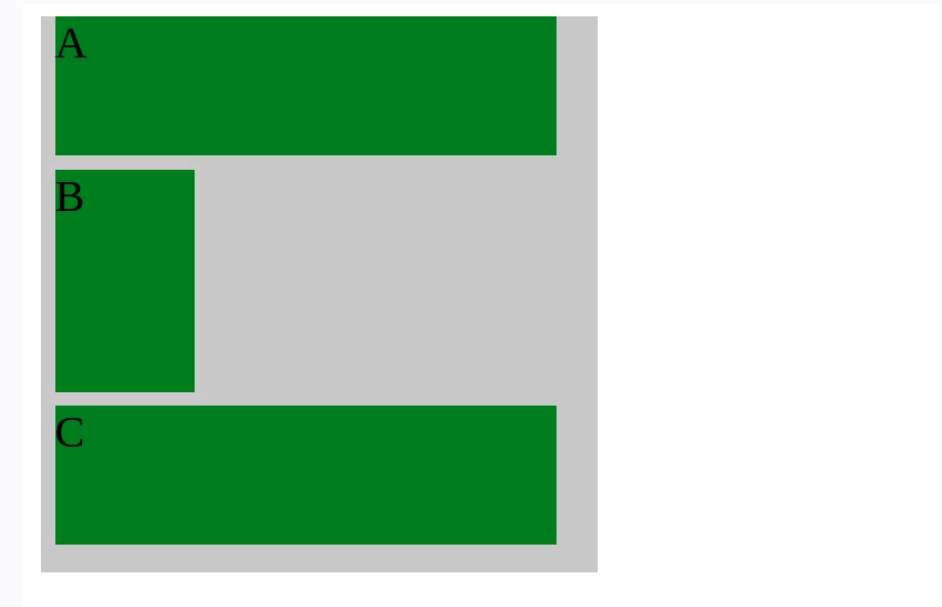
Ejemplo práctico, vemos que cada caja tiene un comportamiento de block haciendo que las otras cajas se vayan hacia abajo, es decir cada caja ocupa una fila completa, aunque el ancho sea de 100px.

💡 **Ejercicio práctico:** Crea tres `<div>` con `display: block` y observa cómo se apilan uno debajo del otro, ocupando todo el ancho disponible.

Ejemplo práctico

```
<section>
  <h2>Display block</h2>
  <div class="contenedor">
    <div class="caja_block">A</div>
    <div class="caja_block caja_block1">B</div>
    <div class="caja_block">C</div>
  </div>
</section>
```

```
.contenedor {
  width: 200px;
  height: 200px;
  background-color: #ccc; /* Fondo gris */
}
.caja_block {
  /* Cada div ocupa toda la fila disponible */
  width: 180px;
  height: 50px;
  margin: 5px; /* Separación entre cajas */
  background-color: rgb(107, 150, 107); /* Verde */
}
.caja_block1 {
  height: 80px; /* Caja más alta */
  width: 50px; /* Más angosta */
}
```



Display: Inline

Características principales

- Se alinean en la misma línea horizontal
- Solo ocupan el espacio de su contenido
- No aceptan propiedades de ancho y alto
- Los márgenes verticales no se aplican correctamente

Ejemplos comunes

- Enlaces `<a>`
- Texto en negrita `<strong>`
- Texto enfatizado `<em>`
- Elementos `<span>`

```
<section>
  <h2>Display Inline</h2>
  <div class="contenedor_inline">
    <div class="caja_inline">A</div>
    <div class="caja_inline">Bbb</div>
    <div class="caja_inline">Cccccccccccccc</div>
  </div>
</section>
```

```
.contenedor_inline {
  width: 200px;
  height: 50px;
  background-color: #ccc;
}
.caja_inline {
  display: inline; /* Ocupa solo el ancho de su contenido */
  background: lightgreen;
  margin: 5px; /* Ojo: margin vertical casi no se aplica en inline */
}
```

Ejemplo práctico, forzamos a través de css un div que tiene por defecto el comportamiento de block a inline.



💡 **Ejercicio práctico:** Cambia un párrafo a `display: inline` y observa cómo se comporta junto a otros elementos.

Display: Inline-Block

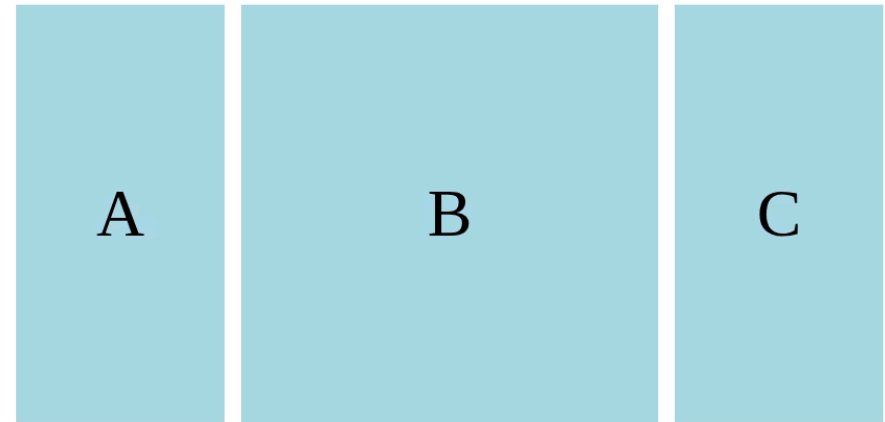
Esta propiedad combina lo mejor de `block` e `inline`, permitiendo que los elementos:

- 1 Se alineen en la misma línea horizontal (como `inline`)
- 2 Acepten propiedades de ancho, alto, márgenes y padding (como `block`)
- 3 Mantengan su integridad como bloque al aplicar estilos

Es ideal para crear menús de navegación, galerías de imágenes o botones alineados horizontalmente.

```
<section>
  <h2>Display Inline block</h2>
  <div class="contenedor_inline_block">
    <div class="caja_inline_block">A</div>
    <div class="caja_inline_block caja1_inline_block">B</div>
    <div class="caja_inline_block">C</div>
  </div>
</section>
```

Ejemplo práctico



```
.contenedor_inline_block {
  width: 300px;
  height: 150px;
}
.caja_inline_block {
  display: inline-block; /* Se comporta como inline, pero respeta
alto y ancho */
  width: 50px;
  height: 100px;
  background-color: lightblue;
  text-align: center; /* Centra el texto horizontalmente */
  line-height: 100px; /* Centra el texto verticalmente */
}
.caja1_inline_block {
  height: 150px;
  width: 100px; /* Caja más grande para notar la diferencia */
}
```

💡 **Ejercicio práctico:** Crea varios botones con `inline-block` para que se alineen horizontalmente mientras mantienes control sobre sus dimensiones.

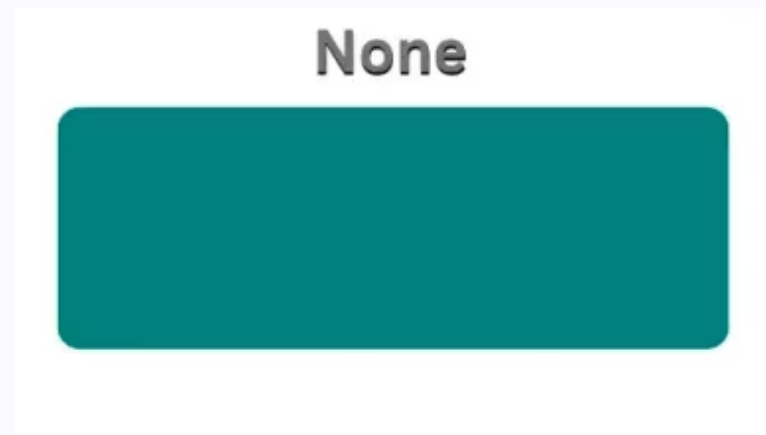
Display: None

Cuando aplicas `display: none` a un elemento:

- El elemento desaparece completamente del flujo del documento
- No ocupa ningún espacio en la página
- Es como si el elemento no existiera en el HTML
- No es accesible para lectores de pantalla

Esta propiedad es útil para:

- Crear menús desplegables
- Mostrar/ocultar contenido según interacciones
- Implementar pestañas de contenido



```
.none {  
  display: none; /* El elemento desaparece y NO  
  ocupa espacio */  
}
```

💡 **Ejercicio práctico:** Crea un botón que oculte/muestre un párrafo cambiando `display: none` con JavaScript.

Visibility: Hidden vs Display: None

visibility: hidden

- El elemento se oculta pero **sigue ocupando espacio** en la página
- El elemento está técnicamente presente pero invisible
- Ideal cuando quieres mantener el layout intacto

display: none

- El elemento **desaparece completamente** del flujo del documento
- No ocupa ningún espacio
- El layout se reajusta como si el elemento no existiera

```
<section>
  <h2>Display none vs visibility hidden</h2>
  <div>
    <div class="none"></div>
    <div class="hidden"></div>
  </div>
</section>
```

```
.none {
  display: none; /* El elemento desaparece y NO ocupa espacio */
}
.hidden {
  visibility: hidden; /* El elemento no se ve, pero sigue ocupando espacio */
}
```

💡 **Ejercicio práctico:** Muestra 3 párrafos, oculta el del medio con `visibility: hidden` y observa cómo se mantiene el espacio vacío.

Opacity: Transparencia en CSS

La propiedad `opacity` controla la transparencia de un elemento y todos sus descendientes:

- `opacity: 0`: Completamente transparente (invisible)
- `opacity: 0.5`: Semi-transparente (50%)
- `opacity: 1`: Completamente opaco (normal)

A diferencia de `visibility: hidden`, los elementos con `opacity: 0`:

- Siguen siendo interactivos (se pueden hacer clic)
- Mantienen su espacio en el layout
- Pueden animarse suavemente

Ejemplo práctico



```
.contenedor_opacity {  
  width: 280px;  
  height: 90px;  
  background-color: #ccc;  
}  
.contenedor_opacity div {  
  display: inline-block;  
  width: 80px;  
  height: 80px;  
  background-color: blue;  
}  
.caja1_opacity {  
  opacity: 1; /* Opacidad completa */  
}  
.caja2_opacity {  
  opacity: 0.5; /* Semitransparente */  
}  
.caja3_opacity {  
  opacity: 0.2; /* Casi invisible */  
}
```

```
<section>  
  <h2>Transparencia en CSS</h2>  
  <div class="contenedor_opacity">  
    <div class="caja1_opacity"></div>  
    <div class="caja2_opacity"></div>  
    <div class="caja3_opacity"></div>  
  </div>  
</section>
```

💡 **Ejercicio práctico:** Crea un div y anima su `opacity` de 0 a 1 en 2 segundos con CSS `transition`.

Z-index: Control del Apilamiento

¿Qué hace z-index?

Controla el orden de apilamiento vertical de los elementos posicionados, determinando cuáles aparecen delante o detrás de otros cuando se superponen.

Requisitos para funcionar

Solo funciona en elementos que tienen una posición distinta a static (predeterminada). Debe usarse con position: relative, absolute, fixed o sticky.

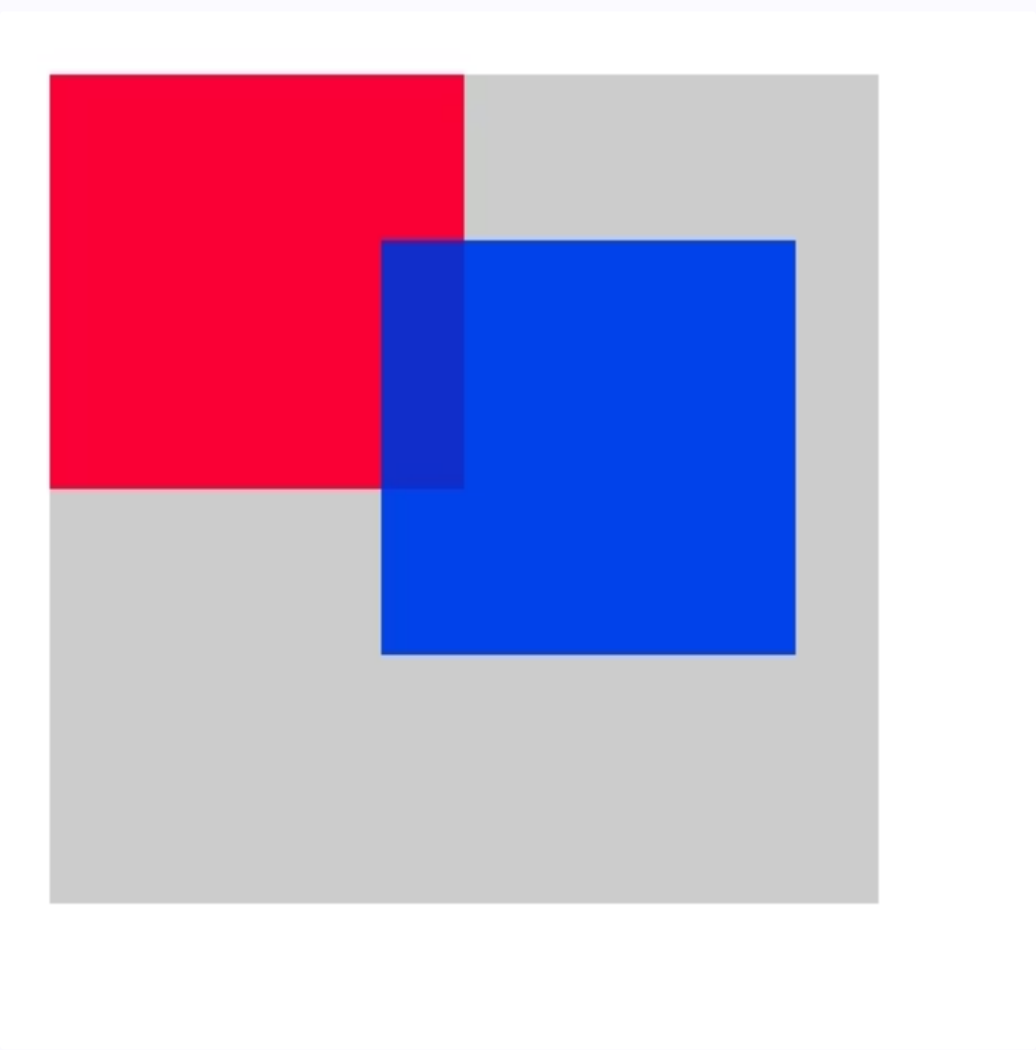
Valores

Acepta números enteros (positivos o negativos). Cuanto mayor sea el número, más "arriba" aparecerá el elemento. Los valores negativos colocan elementos "detrás" del flujo normal.

Ejemplo práctico:

```
<section>
  <h2>Zindex</h2>
  <div class="caja_zIndex">
    <div class="caja1_zIndex"></div>
    <div class="caja2_zIndex"></div>
  </div>
</section>
```

Como deberia verse



```
.caja_zIndex {
  position: relative; /* Necesario para posicionar hijos con absolute */
  background-color: #ccc;
  width: 100px;
  height: 100px;
}
.caja1_zIndex {
  position: absolute; /* Se coloca encima del flujo normal */
  background-color: red;
  width: 50px;
  height: 50px;
  z-index: 2; /* Prioridad de apilamiento */
  opacity: 0.8;
}
.caja2_zIndex {
  background-color: blue;
  position: absolute;
  top: 20px; /* Lo movemos un poco hacia abajo */
  left: 40px; /* Lo desplazamos a la derecha */
  opacity: 0.8;
  width: 50px;
  height: 50px;
  z-index: 4; /* Aparece por encima del rojo */
}
```

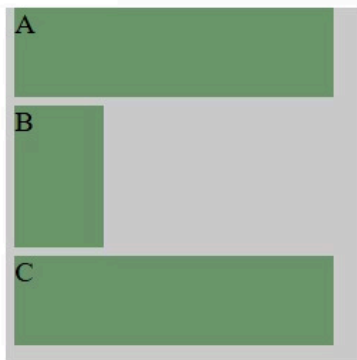
💡 **Ejercicio práctico:** Superpón dos cajas y juega con el z-index para controlar cuál aparece al frente.

¡Tu Css en Acción!

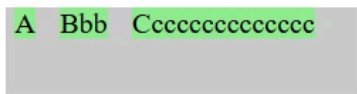
Ahora si hemos seguido todos los pasos correctamente nos daremos cuenta que la web ha quedado increíble con cada una de nuestras secciones dentro del main a la izquierda veremos lo primero y en la segunda el final!

Display y Visibilidad en CSS

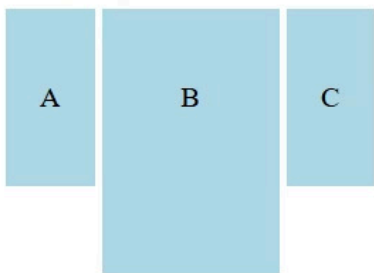
Display block



Display Inline



Display Inline block

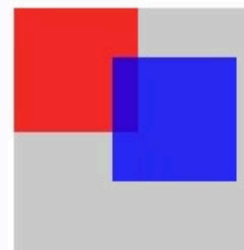


Display none vs visibility hidden

Transparencia en CSS



Zindex



✅ ¡Felicidades! Has completado con éxito esta sección. Tu pagina se ve genial y estás un paso más cerca de dominar el diseño web.